

For example, we cannot apply a modulus operation on complex numbers.

```
>>> (2+2j)%(1+2j)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: can't mod complex numbers.
```

Amongst the arithmetic operations that are allowed, each of them has a typical behavior, depending on the input numeric type and the operation.

A valid complex arithmetic with any numeric type will yield a complex type.

- `>>> 2 * (1+1j)`
- `(2+2j)`

Any operations of a `float` type with an `int` type will result in a `float` value. But in the case of a division operation, the output will be `float` irrespective of input being `float` or `int`. In all other cases, `int` operations yield an `int` value and `float` operations likewise.

- `>>> 2 + 3` #both operands are int
- `5` #result is int
-
- `>>> 2.0 + 3.0` #both operands are float
- `5.0` #result is float
-
- `>>> 2 + 3.0` #one operand is int, other is float
- `5.0` #result is float
-
- `>>> 4 / 2` #both operands are int, but a division operation
- `2.0` #result is float

Binary, Octal and Hexadecimal Numbers

It also provides support for various number systems apart from the decimal number system which is the default for all operations. Other number systems can be easily used with appropriate prefixes. A binary representation is prefixed with `0b` or `0B`, an octal with `0o` or `0O` and a hex using `0x` or `0X`.

- `>>> 0b111` #binary(111) = decimal(7)